DATA STRUCTURES

Complete MCA Exam-Oriented Notes

Unit-wise topics • Simple 2–3 line explanations • Algorithms • Complexity • Exam preparation

UNIT 1 — INTRODUCTION, ARRAYS AND LINKED LISTS

Data Structure

A data structure organizes and stores data so operations such as access, insertion, deletion, and searching can be performed efficiently. Data structures are broadly linear or non-linear.

Abstract Data Type (ADT)

An ADT defines data and the operations allowed on it without specifying implementation details. Stack, queue, and list are common ADTs.

Algorithm and Complexity

An algorithm is a finite sequence of steps used to solve a problem. Time complexity measures growth of running time, while space complexity measures extra memory usage.

Asymptotic Notations

Big-O represents an upper growth bound, Omega a lower bound, and Theta a tight bound. Common complexities are $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, and $O(n^2)$.

Arrays

An array stores elements of the same type in contiguous memory. Indexed access is normally $O(1)$, while middle insertion/deletion may require shifting elements.

Array Operations

Traversal visits elements; insertion adds an element; deletion removes one; searching locates a value; updating changes an existing value. Cost depends on position and implementation.

Sparse Matrix

A sparse matrix contains mostly zero values. Storing only non-zero elements saves memory; triplet representation is a common method.

Linked List

A linked list contains nodes connected by links. Nodes need not be contiguous, making insertion and deletion flexible when the required position/node is known.

Singly Linked List

Each node stores data and a pointer to the next node. Traversal is forward only.

Doubly Linked List

Each node has previous and next pointers, allowing two-way traversal. It uses more memory than a singly linked list.

Circular Linked List

The last node points back to the first node. It is useful for cyclic processing such as round-robin scheduling.

UNIT 2 — STACKS, QUEUES AND RECURSION

Stack

A stack follows LIFO: the last inserted item is removed first. Main operations are push, pop, peek/top, and isEmpty.

Stack Applications

Stacks are used for function calls, recursion, expression conversion/evaluation, undo operations, and backtracking.

Infix, Prefix and Postfix

Infix places operators between operands; prefix before operands; postfix after operands. Prefix/postfix reduce dependence on parentheses.

Recursion

Recursion occurs when a function calls itself on a smaller problem. A base case is required to stop recursion.

Queue

A queue follows FIFO: the first inserted item is removed first. Enqueue adds at the rear and dequeue removes from the front.

Circular Queue

A circular queue connects the last position back to the first and reuses freed spaces efficiently.

Deque

A double-ended queue permits insertion and deletion at both ends.

Priority Queue

A priority queue removes elements according to priority. Heaps are a common efficient implementation.

UNIT 3 — TREES

Tree

A tree is a hierarchical non-linear structure of nodes and edges. It has a root and parent-child relationships.

Binary Tree

A binary tree has at most two children per node, called left and right.

Tree Terminology

Root, parent, child, sibling, leaf, internal node, degree, level, height, and subtree are important tree terms.

Tree Traversals

Preorder is Root-Left-Right, inorder is Left-Root-Right, and postorder is Left-Right-Root. Level order visits nodes level by level using a queue.

Binary Search Tree

A BST stores smaller keys in the left subtree and larger keys in the right subtree. Search, insertion, and deletion are efficient when the tree is balanced.

BST Operations

Insertion follows the ordering rule; deletion handles leaf, one-child, and two-child cases. A skewed BST can have O(n) operations.

AVL Tree

An AVL tree is a self-balancing BST where each node has balance factor -1, 0, or +1. Rotations restore balance.

Heap

A heap is a complete binary tree satisfying a heap property. Max-heaps keep the largest key at the root; min-heaps keep the smallest.

Heap Operations

Insertion adds a node and moves it upward if necessary. Root deletion replaces the root and restores the heap using heapify.

Expression Tree

Operators are internal nodes and operands are leaves. Traversals can represent the expression in prefix, infix, or postfix form.

UNIT 4 — GRAPHS

Graph

A graph contains vertices and edges representing relationships. It can be directed/undirected and weighted/unweighted.

Adjacency Matrix

A matrix stores whether/how vertices are connected. Edge lookup is fast, but space is $O(V^2)$.

Adjacency List

Each vertex stores its neighboring vertices. It is generally more space-efficient for sparse graphs.

BFS

Breadth First Search explores level by level using a queue. It is useful for shortest paths in unweighted graphs.

DFS

Depth First Search explores deeply before backtracking. It can use recursion or a stack.

Spanning Tree

A spanning tree connects all vertices of a connected graph with exactly V-1 edges and no cycles.

Minimum Spanning Tree

An MST is a spanning tree with minimum total edge weight. Prim's and Kruskal's algorithms are common approaches.

Prim's Algorithm

Prim grows an MST from a starting vertex by repeatedly choosing the minimum-weight edge connecting the tree to a new vertex.

Kruskal's Algorithm

Kruskal sorts edges by weight and adds the smallest edge that does not create a cycle. Union-Find is commonly used.

Dijkstra's Algorithm

Dijkstra finds shortest paths from one source when edge weights are non-negative. It repeatedly selects the closest unvisited vertex and relaxes edges.

UNIT 5 — SEARCHING, SORTING AND HASHING

Linear Search

Linear search checks items one by one. Worst-case time complexity is O(n).

Binary Search

Binary search repeatedly halves a sorted search range. Its time complexity is O(log n).

Bubble Sort

Adjacent elements are repeatedly compared and swapped. Its usual time complexity is $O(n^2)$.

Selection Sort

The minimum element of the unsorted portion is selected and placed at its correct position. It uses $O(n^2)$ comparisons.

Insertion Sort

Elements are inserted into an already sorted portion. It is useful for small or nearly sorted data.

Merge Sort

The list is divided, recursively sorted, and merged. Time complexity is O(n log n), with extra merging space.

Quick Sort

A pivot partitions the array into smaller parts which are recursively sorted. Average complexity is O(n log n), worst case $O(n^2)$.

Heap Sort

A heap is built and root elements are repeatedly removed. Time complexity is O(n log n).

Hashing

A hash function maps a key to a table index. With good hashing and collision handling, average search/insertion/deletion can be O(1).

Collision Resolution

Collisions can be handled using separate chaining or open addressing such as linear probing, quadratic probing, and double hashing.

UNIT 6 — ADVANCED STRUCTURES AND ALGORITHM TECHNIQUES

Divide and Conquer

A problem is divided into smaller subproblems, solved recursively, and combined. Merge sort is a classic example.

Greedy Method

A greedy algorithm repeatedly chooses a locally best option. Prim, Kruskal, and suitable shortest-path problems use greedy ideas.

Dynamic Programming

DP stores results of overlapping subproblems to avoid repeated work. It can use memoization or tabulation.

Backtracking

Backtracking builds a solution incrementally and abandons a partial solution when it cannot lead to a valid answer. N-Queens is a classic example.

Disjoint Set / Union-Find

It maintains groups and supports Find and Union operations efficiently. It is widely used in Kruskal's algorithm.

Trie

A trie stores strings character by character and supports efficient prefix searching and autocomplete.

B-Tree

A B-tree is a balanced multiway search tree designed to reduce storage-system/disk I/O by keeping many keys in each node.

B+ Tree

B+ trees keep record pointers/data mainly at leaf nodes and link leaves for efficient ordered and range queries.

IMPORTANT COMPLEXITIES — QUICK REVISION

Topic	Average / Typical	Worst Case
Array access	O(1)	O(1)
Linked-list search	O(n)	O(n)
Stack push/pop	O(1)	O(1)
Queue enqueue/dequeue	O(1)	O(1)
BST search	O(log n)	O(n)
AVL search	O(log n)	O(log n)
BFS / DFS	O(V+E)	O(V+E)
Linear search	O(n)	O(n)
Binary search	O(log n)	O(log n)
Merge sort	O(n log n)	O(n log n)
Quick sort	O(n log n)	$O(n^2)$
Heap sort	O(n log n)	O(n log n)
Hash table search	O(1) avg.	O(n)

IMPORTANT EXAM QUESTIONS

1. Define data structure and explain linear and non-linear structures.
2. Explain Big-O, Omega, and Theta with examples.
3. Compare arrays and linked lists.
4. Explain singly, doubly, and circular linked lists.
5. Explain stack operations and convert an infix expression to postfix.
6. Explain queue, circular queue, deque, and priority queue.
7. Explain binary tree traversals with an example.
8. Construct a BST from a given sequence and perform traversal/deletion.
9. Explain AVL trees and LL, RR, LR, and RL rotations.
10. Explain heap construction and insertion/deletion.
11. Compare BFS and DFS with applications and complexity.
12. Find an MST using Prim's or Kruskal's algorithm.
13. Apply Dijkstra's algorithm to a weighted graph.
14. Compare linear and binary search.
15. Perform sorting using Merge, Quick, Heap, Insertion, Selection, or Bubble Sort.
16. Explain hashing and collision-resolution methods.
17. Explain divide and conquer, greedy, dynamic programming, and backtracking.
18. Explain trie, B-tree, and B+ tree.